



**ECOLE MAROCAINE DES
SCIENCES DE L'INGENIEUR**
Membre de
HONORIS UNITED UNIVERSITIES



GESTION DE STOCK

Projet Django



SOUTENUPAR:

Meskini Saad – Sifeddine Oualhane – Rayane Bendaanoune

ENCADRER PAR:

Hamza Abouabid

ANNÉE UNNIVERSITAIRE : 2025-2026

Dédicaces

Je dédie ce modeste travail, fruit de plusieurs mois d'apprentissage, d'efforts et de persévérance, à toutes les personnes qui m'ont soutenu et encouragé tout au long de mon parcours académique.

À mes très chers parents, aucun mot ne pourra exprimer pleinement ma profonde gratitude et mon immense respect envers eux. Leur amour, leurs sacrifices, leur patience et leurs encouragements constants ont toujours été ma principale source de motivation. Grâce à leur soutien moral et leurs conseils précieux, j'ai pu poursuivre mes études dans les meilleures conditions et surmonter les difficultés rencontrées durant mon parcours.

À mes frères et à toute ma famille, pour leur soutien, leur confiance et leurs encouragements permanents. Leur présence à mes côtés m'a donné la force et la motivation nécessaires pour continuer à avancer et atteindre mes objectifs.

À mes enseignants et encadrants, pour leur accompagnement, leur disponibilité et les connaissances qu'ils nous ont transmises tout au long de notre formation. Leurs conseils et leurs orientations ont grandement contribué à la réussite de ce projet.

À mes amis et collègues, pour leur aide, leurs encouragements et tous les moments de collaboration et de partage qui ont rendu cette expérience plus enrichissante et motivante.

Enfin, je dédie ce travail à toute personne ayant contribué, de près ou de loin, à la réalisation de ce projet, ainsi qu'à tous ceux qui m'ont soutenu durant mon parcours académique et personnel.

Que ce travail soit le témoignage de ma reconnaissance et de mon profond respect envers toutes ces personnes.

Remerciements

Le système développé permet d'améliorer l'organisation des données et de simplifier les opérations de gestion du stock. Grâce à l'utilisation du framework Django, le développement de l'application a été plus structuré, sécurisé et maintenable. Nous tenons à exprimer nos sincères remerciements à toutes les personnes qui ont contribué, de près ou de loin, à la réalisation de ce projet de fin d'année.

Nous adressons tout d'abord nos remerciements à notre établissement pour la qualité de la formation dispensée, ainsi que pour les moyens pédagogiques et techniques mis à notre disposition, qui nous ont permis de mener à bien ce projet dans de bonnes conditions.

Nous tenons à remercier particulièrement notre encadrant MR HAMZA ABOUABID pour son accompagnement tout au long de ce travail. Sa disponibilité, ses conseils et son orientation nous ont été d'une grande aide pour structurer notre démarche, surmonter les difficultés rencontrées et améliorer la qualité de notre travail.

Nous remercions également l'ensemble de nos enseignants pour les connaissances et les compétences qu'ils nous ont transmises durant notre parcours. Leur encadrement et leur engagement ont largement contribué à notre formation et à la réalisation de ce projet.

Nos remerciements s'adressent aussi à toutes les personnes qui nous ont apporté leur aide, que ce soit par leurs conseils, leurs remarques ou leurs encouragements, et qui ont contribué à l'enrichissement de ce travail.

Enfin, nous exprimons notre profonde gratitude à nos familles pour leur soutien constant, leur patience et leurs encouragements tout au long de nos études et particulièrement durant la réalisation de ce projet.

Table des matières

Table des matières

Dédicaces.....	0
Remerciements.....	2
Table des matières	3
Introduction générale	5
Chapitre 1 : Analyse des Besoins et Spécifications.....	6
Introduction.....	6
Analyse Détaillée du Cahier des Charges	6
Fonctionnalités Attendues	7
Gestion des utilisateurs	7
Gestion des produits	7
Gestion des commandes fournisseurs.....	7
Gestion des mouvements de stock	7
Alertes et suggestions	7
•Analyses et rapports.....	7
Administration	8
Spécification des besoins non fonctionnels	8
Chapitre 2 : Conception de système	9
Introduction.....	9
Présentation des cas d'utilisation	9
Présentation des acteurs.....	9
Description des cas d'utilisation.....	9
Diagramme des cas d'utilisation.....	10
Description du diagramme de classes	11
Classe Utilisateur	11
Classe Produit	11
Classe Categorie	12
Classe Fournisseur	12
Classe Commande	13
Classe LigneCommande	13
Classe Mouvement_Stock	14
Diagramme de classe	15
Description du diagramme de séquence	16
Étapes du processus.....	16
Diagramme de séquence.....	18
Description des principaux modules et composants	19
Architecture logicielle.....	19
Conclusion	19

Chapitre 3 : Réalisation et Développement	20
Implémentation	20
Développement Backend	21
Développement Frontend.....	25
Gestion des données et API	28
Base de données	32
Interfaces Graphiques	35
Interface de connexion.....	35
Tableau de bord	36
Gestion des produits	37
Gestion des mouvements de stock	39
Tests	40
Tests fonctionnels.....	40
Tests de sécurité.....	41
Tests de performance	42
Chapitre 5 : Conclusion.....	43
Conclusion générale	43
Perspectives.....	44
Bibliographie	45
Sites web.....	45

Introduction générale

Dans le contexte actuel, les entreprises et les organisations sont confrontées à des défis croissants en matière de gestion des stocks. Une mauvaise gestion peut entraîner des ruptures de stock, des surstocks coûteux ou des pertes financières importantes. Selon les données récentes, une gestion optimisée des stocks peut réduire les coûts opérationnels de 20 à 30 %. Face à ce constat, le projet vise à développer une application web innovante de gestion de stock, permettant de suivre en temps réel les produits, les mouvements, les commandes fournisseurs et les alertes de réapprovisionnement.

La problématique centrale à laquelle ce projet tente de répondre est la suivante : Comment concevoir et implémenter une plateforme web qui permet une gestion efficace des stocks, avec des alertes automatiques, des suggestions de réapprovisionnement, une valorisation précise et une analyse de rotation, tout en assurant une expérience utilisateur intuitive ?

Ce rapport détaille les différentes étapes de la conception et du développement de l'application de gestion de stock. Dans un premier temps, nous présentons une analyse approfondie des besoins et des spécifications fonctionnelles. Ensuite, nous décrivons l'architecture de l'application en mettant en évidence les choix technologiques. La troisième partie est consacrée à la présentation des fonctionnalités de la plateforme. Nous abordons également les aspects liés aux tests et à la validation.

Enfin, nous concluons en dressant un bilan du projet et en proposant des perspectives d'amélioration.

Chapitre 1 : Analyse des Besoins et Spécifications

Introduction

L'analyse des besoins et des spécifications est une étape cruciale dans le cycle de développement d'un projet informatique. Elle permet de comprendre en profondeur les exigences des utilisateurs et de définir les fonctionnalités et les caractéristiques techniques de la solution à mettre en œuvre.

Analyse Détaillée du Cahier des Charges

Le cahier des charges précise les fonctionnalités clés de l'application de gestion de stock. Elle inclut une gestion complète des utilisateurs avec des profils différenciés (administrateur, gestionnaire, employé), une gestion dynamique des produits permettant leur création, modification et suppression, ainsi qu'un système de suivi des mouvements (entrées et sorties). L'application intègre également des outils d'alerte automatique lorsque le stock descend en dessous d'un seuil critique, des suggestions de réapprovisionnement, une valorisation du stock et une analyse de rotation des produits.

Fonctionnalités Attendues

Gestion des utilisateurs

- Inscription et connexion des utilisateurs
- Profils différenciés : administrateur, gestionnaire, employé
- Tableau de bord personnalisé selon le rôle
- Gestion des permissions et des accès

Gestion des produits

- Création, modification, suppression de produits
- Informations : nom, quantité, prix unitaire, catégorie, fournisseur, seuil d'alerte
- Catégorisation par type de produit
- Upload de photos et documents

Gestion des commandes fournisseurs

- Création de commandes
- Suivi du statut (en attente, reçue, annulée)
- Lignes de commande avec produits, quantités et prix

Gestion des mouvements de stock

- Enregistrement des entrées et sorties
- Historique complet avec date, utilisateur, quantité
- Validation automatique du stock disponible avant sortie

Alertes et suggestions

- Détection automatique des produits en dessous du seuil
- Génération de suggestions de réapprovisionnement
- Priorisation haute/moyenne/basse
- Validation des suggestions par le gestionnaire

Analyses et rapports

- Valorisation totale du stock
- Analyse de rotation des produits
- Graphiques des quantités
- Tendance des produits (hausse/baisse)

Administration

- Modération des utilisateurs
- Gestion des rôles et permissions
- Consultation des statistiques globales

Spécification des besoins non fonctionnels

La spécification des besoins non fonctionnels porte sur plusieurs aspects essentiels :

- Performance : navigation fluide et temps de réponse rapide, même en cas d'utilisation simultanée par plusieurs utilisateurs.
- Sécurité : mécanismes stricts d'authentification, protection des données, gestion rigoureuse des accès.
- Ergonomie : interface intuitive et responsive, compatible avec ordinateurs, tablettes et smartphones.
- Maintenabilité : architecture modulaire conforme au modèle MVT de Django, facilitant les évolutions futures et la correction rapide des bugs.
- Disponibilité : infrastructure stable minimisant les interruptions, garantissant la continuité du service.

Chapitre 2 : Conception de système

Introduction

La conception de l'application est une étape cruciale qui transforme les besoins et les spécifications en un plan détaillé de la solution logicielle. Cette phase consiste à définir l'architecture, concevoir le modèle de données et élaborer l'interface utilisateur.

Présentation des cas d'utilisation

Présentation des acteurs

L'application de gestion de stock implique trois acteurs principaux :

- **Employé** : Consulte les produits et les mouvements. Peut enregistrer des mouvements simples.
- **Gestionnaire** : Gère les produits, les commandes fournisseurs, valide les suggestions de réapprovisionnement, consulte les analyses et rapports.
- **Administrateur** : Gère les utilisateurs et leurs rôles, modère l'accès aux fonctionnalités, supervise l'ensemble du système.

Description des cas d'utilisation

- **Authentification** : Permet aux utilisateurs de se connecter à la plateforme.
- **Gérer les produits** : Création, modification, suppression et consultation des produits.
- **Gérer les commandes** : Création et suivi des commandes fournisseurs.
- **Enregistrer un mouvement** : Ajout d'une entrée ou d'une sortie de stock.
- **Consulter les alertes** : Visualisation des produits en dessous du seuil critique.
- **Générer des suggestions** : Création automatique de suggestions de réapprovisionnement.
- **Valider une suggestion** : Transformation d'une suggestion en commande fournisseur.
- **Consulter la valorisation** : Visualisation de la valeur totale du stock et du détail par produit.
- **Analyser la rotation** : Consultation du ratio de rotation et des tendances.
- **Gérer les utilisateurs** : Création, modification et suppression des comptes (administrateur).

Diagramme des cas d'utilisation

Le diagramme offre une vue d'ensemble des interactions entre les acteurs (Employé, Gestionnaire, Administrateur) et le système. Il illustre les principales fonctionnalités offertes par la plateforme et montre comment chaque acteur interagit avec ces fonctionnalités.

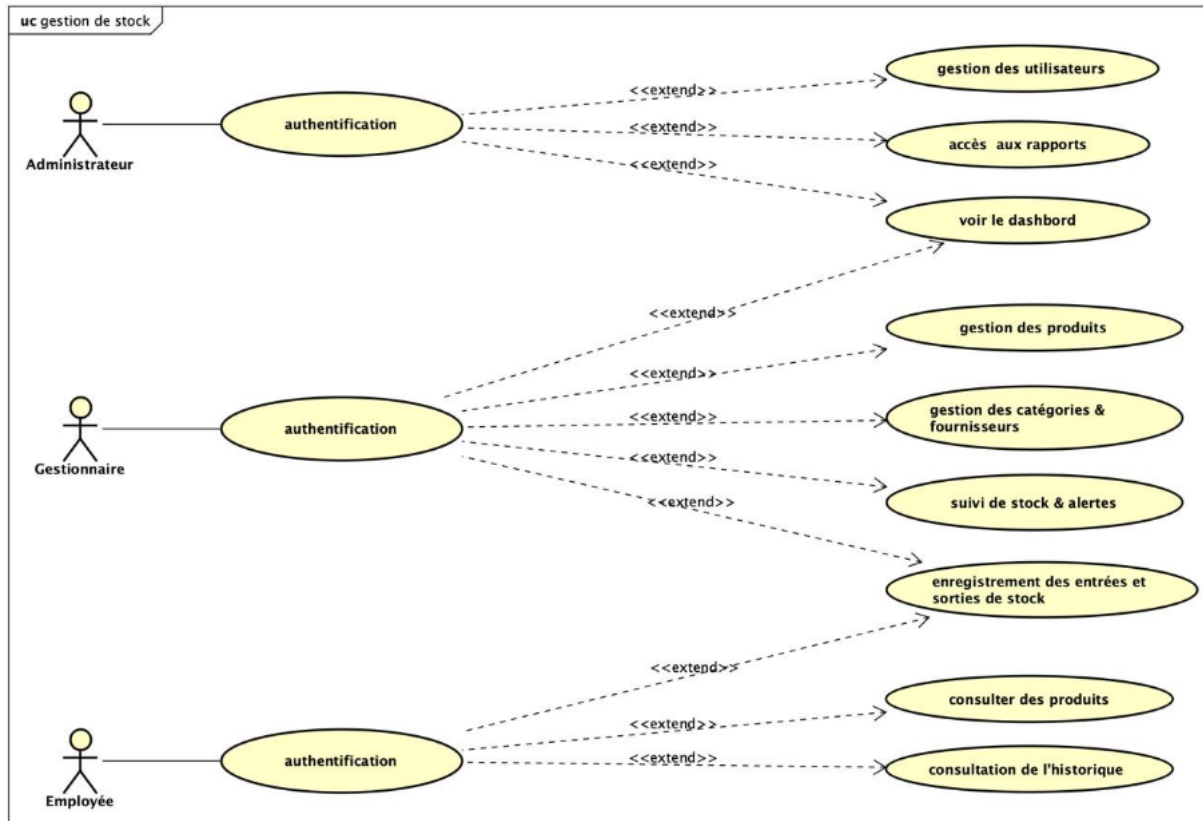


Figure 1 : Diagramme de cas d'utilisation

Description du diagramme de classes

Classe Utilisateur

La classe **Utilisateur** représente les personnes qui utilisent l'application, comme les administrateurs ou les gestionnaires de stock.

Attributs :

- id : identifiant unique de l'utilisateur
- username : nom d'utilisateur
- password : mot de passe
- role : rôle de l'utilisateur

Méthodes :

- seConnecter() : permet à l'utilisateur de se connecter
- seDeconnecter() : permet de quitter la session

Relation :

Un utilisateur peut gérer plusieurs commandes et plusieurs mouvements de stock.

Classe Produit

La classe **Produit** représente les articles disponibles dans le stock.

Attributs :

- id : identifiant du produit
- nom : nom du produit
- description : description du produit
- prix_unitaire : prix du produit
- quantite : quantité disponible
- seuil_alerte : niveau minimal du stock
- code_barre : code-barres du produit

Méthodes :

- ajouterProduit()
- modifierProduit()
- supprimerProduit()

Relation :

- Un produit appartient à une seule catégorie.
- Un produit est fourni par un fournisseur.
- Un produit peut apparaître dans plusieurs lignes de commande.
- Un produit peut avoir plusieurs mouvements de stock.

Classe Catégorie

La classe **Catégorie** permet de classer les produits selon leur type.

Attributs :

- id
- nom

Relation :

Une catégorie peut contenir plusieurs produits, tandis qu'un produit appartient à une seule catégorie.

Classe Fournisseur

La classe **Fournisseur** représente les fournisseurs des produits.

Attributs :

- id
- nom
- telephone
- email
- adresse

Relation :

Un fournisseur peut fournir plusieurs produits et recevoir plusieurs commandes.

Classe Commande

La classe **Commande** représente les commandes effectuées dans le système.

Attributs :

- id
- date_commande
- statut

Méthodes :

- ajouterCommande()
- listesCommande()

Relation :

- Une commande est créée par un utilisateur.
- Une commande contient plusieurs lignes de commande.
- Une commande est liée à un fournisseur.

Classe LigneCommande

La classe **LigneCommande** représente les détails des produits commandés.

Attributs :

- id
- quantite
- prix

Relation :

- Une ligne de commande appartient à une seule commande.

- Une ligne de commande correspond à un seul produit.

Cette classe permet de gérer les produits contenus dans chaque commande.

Classe Mouvement_Stock

La classe **Mouvement_Stock** permet de gérer les entrées et sorties du stock.

Attributs :

- id
- type_mouvement
- quantite
- date_mouvement

Méthode :

- enregistrer()

Relation :

- Chaque mouvement est associé à un produit.
- Chaque mouvement est effectué par un utilisateur.

Diagramme de classe

Ce diagramme de classes représente le système de gestion de stock développé dans le projet. Il illustre les différentes entités du système ainsi que les relations entre elles. L'objectif principal de cette modélisation est d'assurer une gestion efficace des produits, des commandes, des fournisseurs et des mouvements de stock.

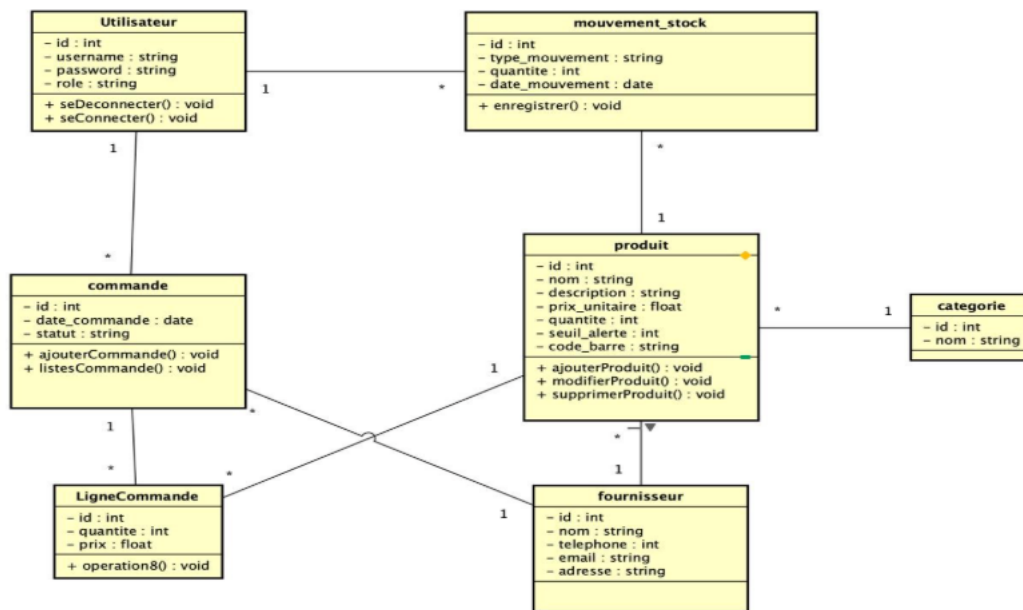


Figure 2 : Diagramme de classe

Description du diagramme de séquence

Ce diagramme de séquence représente le processus d'ajout d'un mouvement de stock dans le système de gestion de stock. Il montre les interactions entre les différents acteurs et composants du système : l'utilisateur, l'interface, le système et la base de données.

Étapes du processus

1. Connexion de l'utilisateur

- L'utilisateur commence par se connecter à l'application.
- L'interface envoie les informations de connexion (login et mot de passe) au système.
- Le système vérifie les informations dans la base de données.
- Si les informations sont correctes, la connexion est acceptée.

2. Choix de l'ajout d'un mouvement de stock

- Après la connexion, l'utilisateur choisit l'option « Ajouter mouvement stock ».
- L'interface demande alors au système d'afficher le formulaire correspondant.
- Le système renvoie le formulaire à l'interface.

3. Saisie des informations

- L'utilisateur saisit les informations nécessaires :
 - le type de mouvement (entrée ou sortie),
 - le produit concerné,
 - la quantité.
- L'interface transmet ensuite ces données au système.

4. Vérification du produit

- Le système vérifie l'existence du produit dans la base de données.
- Si le produit existe, le traitement continue.

5. Mise à jour du stock

- Deux cas sont possibles :
 - **Entrée de stock** : la quantité du produit est augmentée.
 - **Sortie de stock** : la quantité du produit est diminuée.

6. Enregistrement du mouvement

- Le système enregistre le mouvement dans la table des mouvements de stock.
- La base de données confirme l'enregistrement.

7. Confirmation

- Le système envoie un message de confirmation à l'interface.
- L'utilisateur reçoit finalement le message indiquant que le mouvement a été ajouté avec succès.

Diagramme de séquence

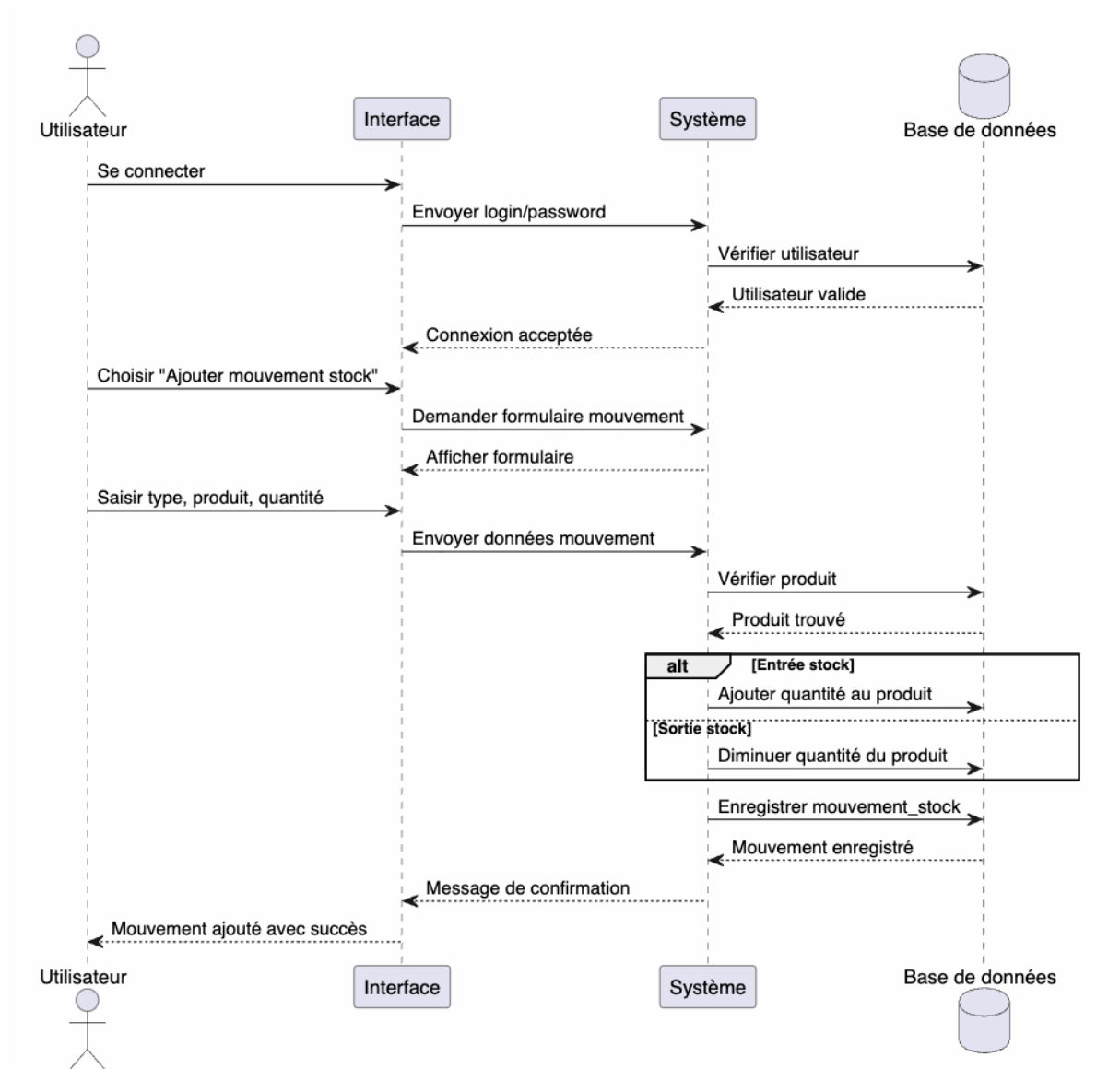


Figure 3 : Diagramme de séquence

Description des principaux modules et composants

- **Module Utilisateurs** : Gestion des comptes, authentification, rôles et permissions.
- **Module Produits** : CRUD des produits, catégories, fournisseurs.
- **Module Commandes** : Gestion des commandes fournisseurs et des lignes de commande.
- **Module Mouvements** : Enregistrement des entrées et sorties, historique.
- **Module Alertes** : Détection automatique des stocks critiques.
- **Module Suggestions** : Génération et gestion des réapprovisionnements.
- **Module Analyses** : Valorisation, rotation, graphiques.

Architecture logicielle

L'architecture de l'application est basée sur le modèle Modèle-Vue-Template (MVT) de Django :

- **Modèles** : Définition de la structure des données et des règles métier.
Implémentation avec l'ORM de Django pour interagir avec la base de données MySQL.
- **Vues** : Gestion de la logique métier. Reçoivent les requêtes HTTP, effectuent les traitements et retournent une réponse (template HTML ou données JSON).
- **Templates** : Interface utilisateur en HTML avec des balises Django pour l'affichage dynamique.
- **Contrôleurs d'URL** : Mapping des URL vers les vues appropriées.
- **Formulaires** : Création et validation des données saisies.
- **Authentification** : Gestion des utilisateurs, mots de passe et permissions.
- **Middleware** : Traitements transversaux (sessions, sécurité).

Conclusion

• Ce chapitre a présenté les choix de conception clés pour l'application de gestion de stock. L'architecture MVT de Django permet une séparation claire des responsabilités. Le modèle de données couvre l'ensemble des besoins : produits, commandes, mouvements, alertes et suggestions. Ces choix servent de base pour la phase de développement décrite dans le chapitre suivant.

Chapitre 3 : Réalisation et Développement

Implémentation

Dans cette partie, nous présentons les différentes étapes de développement de l'application de gestion de stock réalisée avec le framework Django. Cette phase représente l'étape la plus importante du projet, car elle transforme les besoins et les diagrammes réalisés durant la phase de conception en une application fonctionnelle et exploitable.

L'objectif principal de cette implémentation est de développer une plateforme capable de faciliter la gestion quotidienne du stock au sein d'une entreprise ou d'un magasin. Le système permet notamment de gérer les produits, les catégories, les fournisseurs, les mouvements de stock ainsi que les utilisateurs de manière simple, rapide et sécurisée.

Pour assurer le bon fonctionnement du système, nous avons adopté une architecture organisée séparant les différentes parties de l'application. Cette organisation permet une meilleure maintenance du code, une amélioration des performances ainsi qu'une évolution plus simple du projet dans le futur.

Le développement de l'application a été réalisé progressivement en commençant par la conception de la base de données, ensuite le développement du backend, puis la création des interfaces graphiques et enfin les tests de validation.

Développement Backend

Le backend représente la partie serveur de l'application. Il joue un rôle essentiel dans le fonctionnement du système puisqu'il assure le traitement des données, l'exécution des fonctionnalités ainsi que la communication avec la base de données.

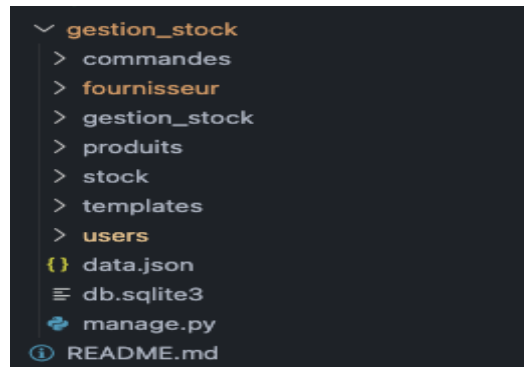


Figure 4 : Structure générale du projet Django

Le développement du backend a été réalisé avec le langage Python en utilisant le framework Django. Ce framework a été choisi grâce à sa robustesse, sa sécurité et sa rapidité de développement. Il offre également plusieurs fonctionnalités intégrées permettant de simplifier la création des applications web modernes.

La gestion des données a été assurée par le système de gestion de base de données MySQL. Ce choix permet de garantir une bonne organisation des informations ainsi qu'une meilleure performance lors des opérations de stockage et de récupération des données.

Le backend développé permet de gérer plusieurs fonctionnalités importantes du système, notamment :

- la gestion des utilisateurs,
- la gestion des produits,
- la gestion des catégories,
- la gestion des fournisseurs,
- la gestion des mouvements de stock,
- l'authentification et la sécurité des accès.

Afin de rendre le projet plus organisé et plus maintenable, l'application a été divisée en plusieurs applications Django indépendantes. Chaque application possède un rôle précis dans le système. Cette approche améliore la lisibilité du code et facilite le développement collaboratif.

- l'application **produits** permet la gestion des articles et des catégories,
- l'application **utilisateurs** gère l'authentification et les permissions,
- l'application **mouvements** permet de suivre les entrées et sorties du stock,
- l'application **fournisseurs** gère les informations des fournisseurs.

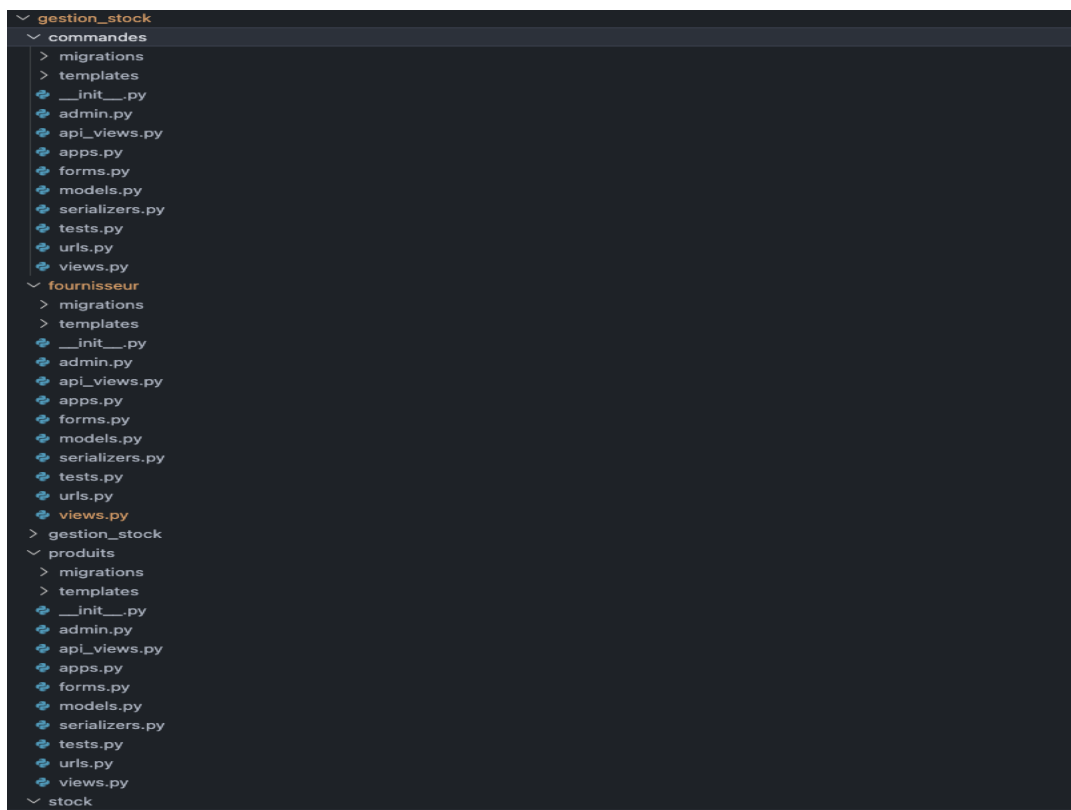


Figure 5: Organisation des applications Django

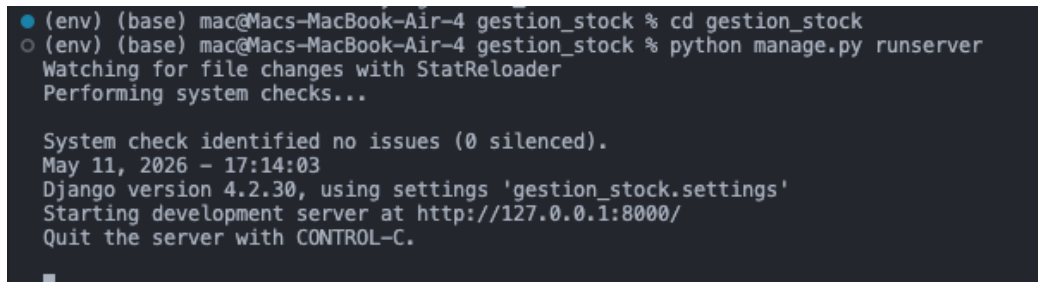
Les modèles Django ont été utilisés afin de représenter les différentes tables de la base de données. Chaque modèle contient les attributs nécessaires pour stocker les informations relatives aux produits, utilisateurs ou mouvements de stock. Grâce au système ORM de Django, les opérations sur la base de données peuvent être réalisées facilement sans écrire directement des requêtes SQL complexes.

Les vues Django ont été utilisées pour traiter les requêtes des utilisateurs et afficher les données dans les différentes pages du système. Les templates HTML permettent quant à eux de construire les interfaces graphiques visibles par les utilisateurs.

```
1 def home(request):
2     if request.user.role == 'EMPLOYEE':
3         return redirect('liste_produits')
4
5     total_produits = Produit.objects.count()
6     produits_alerte = Produit.objects.filter(quantite__lte=F('seuil_alerte')).count()
7     derniers_produits = Produit.objects.all().order_by('-id')[:5]
8
9     # Valorisation du stock
10    valorisation_total = sum(p.valorisation_stock() for p in Produit.objects.all())
11
12    # Produits expirés
13    produits_expires = Produit.objects.filter(date_expiration__lte=timezone.now().date()).count()
14
15    # Produits obsolètes
16    produits_obsolètes = Produit.objects.filter(statut_obsolescence__in=['OBSOLETE', 'DISCONTINU']).count()
17
18    # Suggestions en attente
19    suggestions_en_attente = SuggestionReapprovisionnement.objects.filter(traitee=False).count()
20
21    noms = [p.nom for p in derniers_produits]
22    quantites = [p.quantite for p in derniers_produits]
23
24    context = {
25        'total_produits': total_produits,
26        'produits_alerte': produits_alerte,
27        'produits': derniers_produits,
28        'noms': noms,
29        'quantites': quantites,
30        'valorisation_total': valorisation_total,
31        'produits_expires': produits_expires,
32        'produits_obsolètes': produits_obsolètes,
33        'suggestions_en_attente': suggestions_en_attente,
34    }
35    return render(request, 'home.html', context)
36
```

Figure 6: Gestion des vues et des routes

Le serveur de développement Django a été utilisé durant toute la phase de réalisation afin de tester les fonctionnalités de l'application en temps réel et corriger rapidement les erreurs rencontrées pendant le développement.

A terminal window with a dark background and light-colored text. It shows the execution of Django commands. The first line is a prompt with a blue dot icon, followed by the command 'cd gestion_stock'. The second line is a prompt with a grey circle icon, followed by the command 'python manage.py runserver'. The output shows 'Watching for file changes with StatReloader', 'Performing system checks...', 'System check identified no issues (0 silenced)', the date and time 'May 11, 2026 - 17:14:03', 'Django version 4.2.30, using settings \'gestion_stock.settings\'', 'Starting development server at http://127.0.0.1:8000/', and 'Quit the server with CONTROL-C.'.

```
● (env) (base) mac@Macs-MacBook-Air-4 gestion_stock % cd gestion_stock
○ (env) (base) mac@Macs-MacBook-Air-4 gestion_stock % python manage.py runserver
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).
May 11, 2026 - 17:14:03
Django version 4.2.30, using settings 'gestion_stock.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CONTROL-C.
```

Figure 7: Exécution du serveur Django

Développement Frontend

Le frontend représente la partie visible de l'application avec laquelle les utilisateurs interagissent quotidiennement. Cette interface joue un rôle essentiel dans le système puisqu'elle permet d'assurer une communication simple, rapide et intuitive entre l'utilisateur et l'application de gestion de stock.

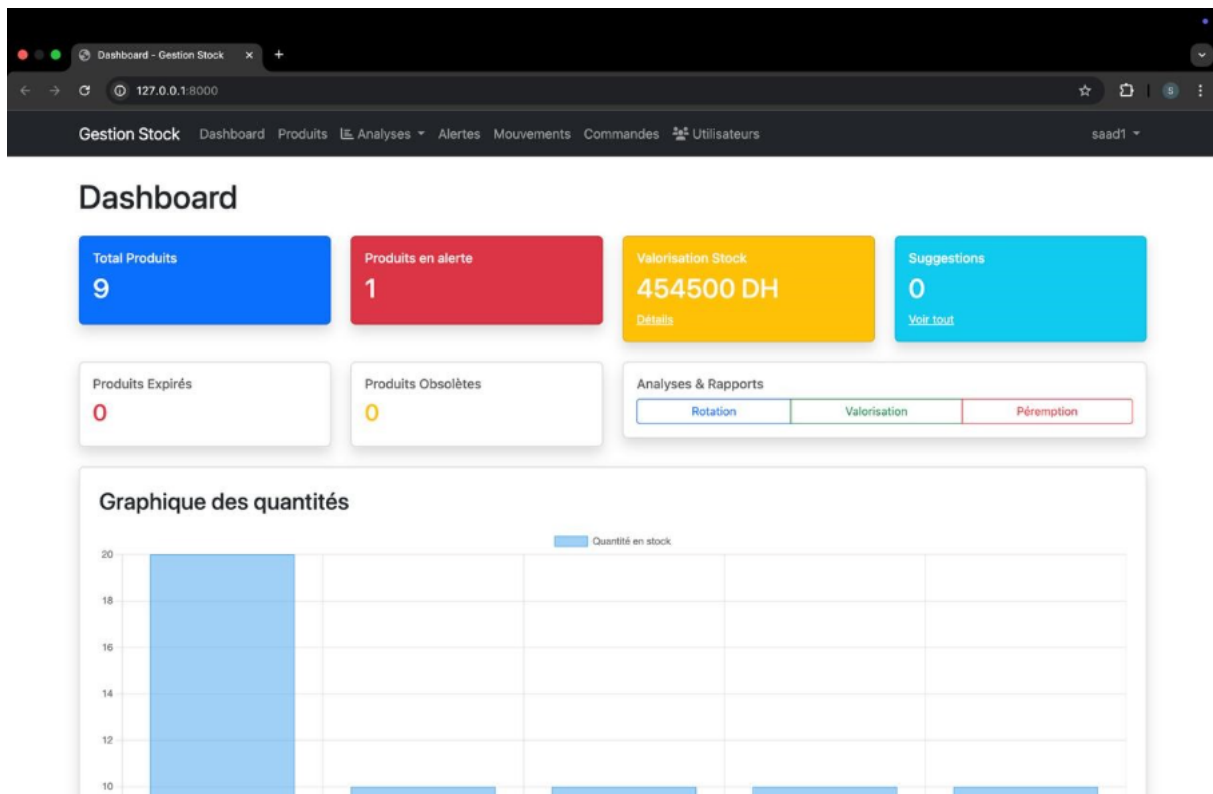


Figure 8: Interface d'accueil du système

Le développement du frontend a été réalisé à l'aide des technologies HTML, CSS et JavaScript. Afin d'améliorer l'apparence graphique et rendre l'interface plus moderne et responsive, nous avons également utilisé le framework Bootstrap. Ce framework facilite la création d'interfaces professionnelles compatibles avec différentes tailles d'écran.

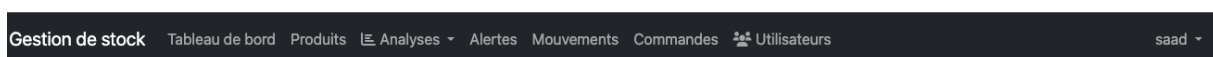


Figure 9: Barre de navigation de l'application

Les templates du framework Django ont été utilisés afin d’afficher dynamiquement les données provenant de la base de données. Grâce au moteur de templates Django, les informations peuvent être affichées automatiquement sans recharger manuellement les pages.

L’interface utilisateur a été conçue de manière ergonomique afin de simplifier l’utilisation du système. Plusieurs pages ont été développées pour assurer les différentes fonctionnalités de gestion du stock.

Le frontend permet notamment :

- la consultation des produits disponibles,
- l’ajout de nouveaux produits,
- la modification des informations des produits,
- la suppression des produits,
- la gestion des mouvements de stock,
- l’affichage des statistiques principales du système.

Une attention particulière a été accordée à l’expérience utilisateur afin de rendre la navigation plus fluide et plus agréable. Les couleurs, les boutons, les tableaux et les formulaires ont été organisés de manière claire afin de faciliter l’utilisation de l’application même pour les utilisateurs débutants.

L'application possède également une interface responsive qui s'adapte automatiquement aux ordinateurs, tablettes et téléphones mobiles. Cette compatibilité améliore l'accessibilité du système et permet son utilisation dans plusieurs environnements.

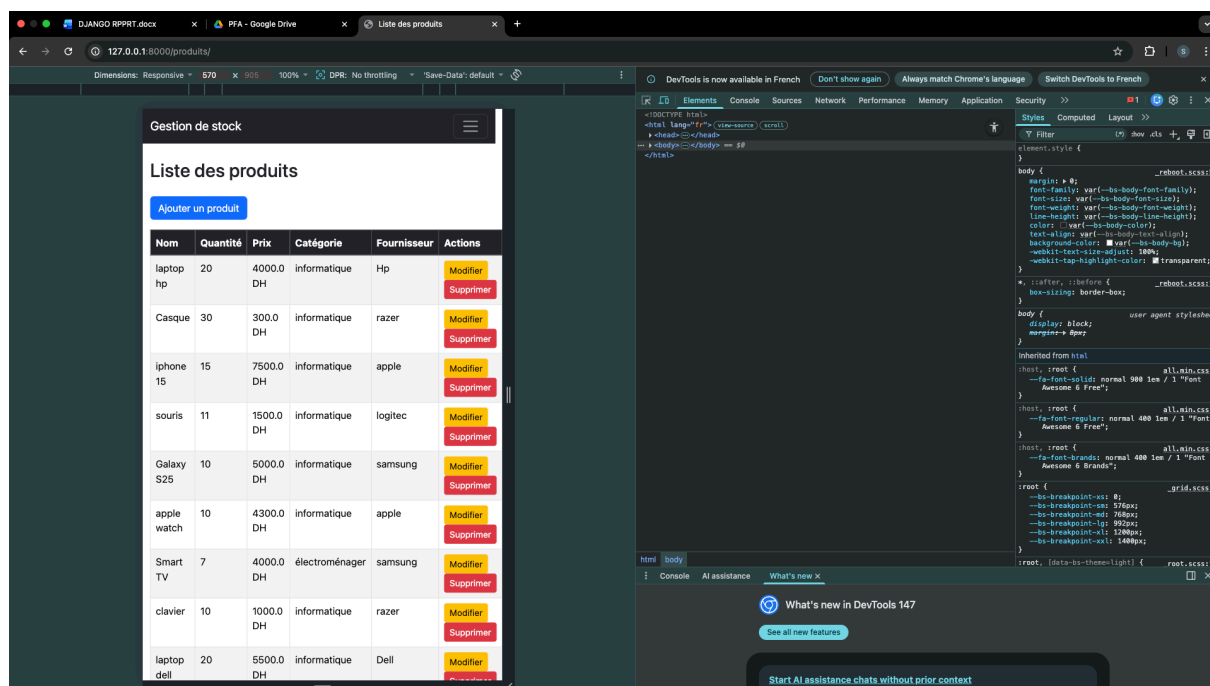


Figure 10 : Interface responsive de l'application

Gestion des données et API

La gestion des données constitue une partie essentielle du système de gestion de stock. Elle permet de manipuler efficacement les informations relatives aux produits, aux fournisseurs, aux utilisateurs ainsi qu'aux mouvements de stock.

Le framework Django offre plusieurs fonctionnalités intégrées facilitant les opérations CRUD (Create, Read, Update, Delete). Ces opérations permettent respectivement d'ajouter, consulter, modifier et supprimer les données du système.



```
1  def ajouter_produit(request):
2  if request.method == 'POST':
3  form = ProduitForm(request.POST)
4  if form.is_valid():
5  form.save()
6  return redirect('liste_produits')
7  else:
8  form = ProduitForm()
9  return render(request, 'produits/form_produit.html', {'form': form})
10
11 @admin_or_gestionnaire_required
12 def modifier_produit(request, id):
13 produit = get_object_or_404(Produit, id=id)
14 if request.method == 'POST':
15 form = ProduitForm(request.POST, instance=produit)
16 if form.is_valid():
17 form.save()
18 return redirect('liste_produits')
19 else:
20 form = ProduitForm(instance=produit)
21 return render(request, 'produits/form_produit.html', {'form': form})
22
23 @admin_or_gestionnaire_required
24 def supprimer_produit(request, id):
25 produit = get_object_or_404(Produit, id=id)
26 if request.method == 'POST':
27 produit.delete()
28 return redirect('liste_produits')
29 return render(request, 'produits/confirmer_suppression.html', {'produit': produit})
30
```

Figure 11: Opérations CRUD dans le système

Les opérations principales réalisées dans l'application sont :

- l'ajout des produits,
- la modification des informations des produits,
- la suppression des produits,
- la consultation des stocks disponibles,
- la gestion des mouvements d'entrée et de sortie.

Les vues Django jouent un rôle important dans le traitement des requêtes envoyées par les utilisateurs. Elles assurent la communication entre les templates HTML et la base de données afin d'afficher les informations correctement dans les différentes interfaces.

Les formulaires Django ont été utilisés afin de simplifier la saisie des informations et garantir leur validation avant l'enregistrement dans la base de données. Cette validation permet de réduire les erreurs et d'améliorer la fiabilité des données enregistrées.

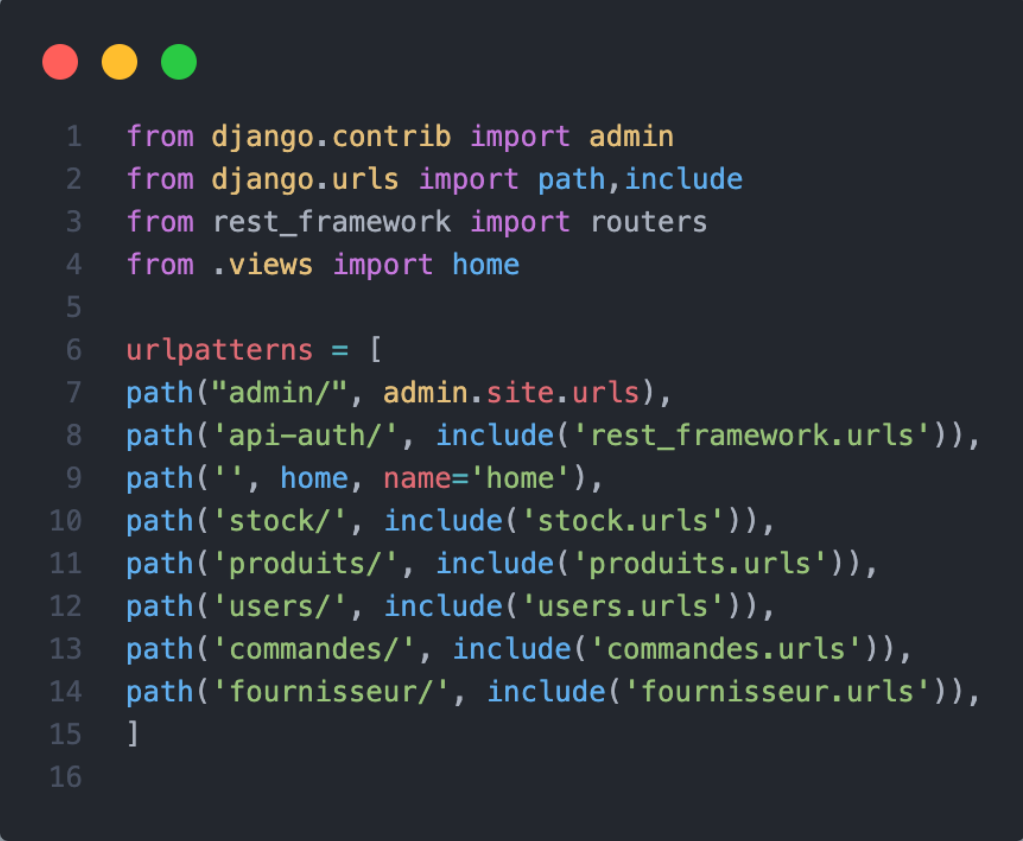
```

1  from django import forms
2  from .models import Produit
3
4  class ProduitForm(forms.ModelForm):
5      class Meta:
6          model = Produit
7          fields = '__all__'
8          widgets = {
9              'nom': forms.TextInput(attrs={'class': 'form-control'}),
10             'description': forms.Textarea(attrs={'class': 'form-control', 'rows': 3}),
11             'quantite': forms.NumberInput(attrs={'class': 'form-control'}),
12             'seuil_alerte': forms.NumberInput(attrs={'class': 'form-control'}),
13             'code_barre': forms.TextInput(attrs={'class': 'form-control'}),
14             'prix': forms.NumberInput(attrs={'class': 'form-control', 'step': '0.01'}),
15             'categorie': forms.Select(attrs={'class': 'form-select'}),
16             'fournisseur': forms.Select(attrs={'class': 'form-select'}),
17             'date_expiration': forms.DateInput(attrs={'class': 'form-control', 'type': 'date'}),
18             'statut_obsolescence': forms.Select(attrs={'class': 'form-select'}),
19             'date_obsolescence': forms.DateInput(attrs={'class': 'form-control', 'type': 'date'}),
20         }
21         labels = {
22             'nom': 'Nom du produit',
23             'description': 'Description',
24             'quantite': 'Quantité en stock',
25             'seuil_alerte': 'Seuil d\'alerte',
26             'code_barre': 'Code-barres',
27             'prix': 'Prix unitaire (DH)',
28             'categorie': 'Catégorie',
29             'fournisseur': 'Fournisseur',
30             'date_expiration': 'Date de péremption',
31             'statut_obsolescence': 'Statut d\'obsolescence',
32             'date_obsolescence': 'Date d\'obsolescence',
33         }
34

```

Figure 12: Formulaires Django

Le système utilise également les routes définies dans le fichier `urls.py` afin d'organiser la navigation entre les différentes pages de l'application. Cette organisation facilite la maintenance et améliore la structure globale du projet.



```
1  from django.contrib import admin
2  from django.urls import path,include
3  from rest_framework import routers
4  from .views import home
5
6  urlpatterns = [
7      path("admin/", admin.site.urls),
8      path('api-auth/', include('rest_framework.urls')),
9      path('', home, name='home'),
10     path('stock/', include('stock.urls')),
11     path('produits/', include('produits.urls')),
12     path('users/', include('users.urls')),
13     path('commandes/', include('commandes.urls')),
14     path('fournisseur/', include('fournisseur.urls')),
15 ]
16
```

Figure 13: Configuration des routes du système

Base de données

La base de données représente l'élément principal du système puisqu'elle assure le stockage et l'organisation des informations utilisées dans l'application.

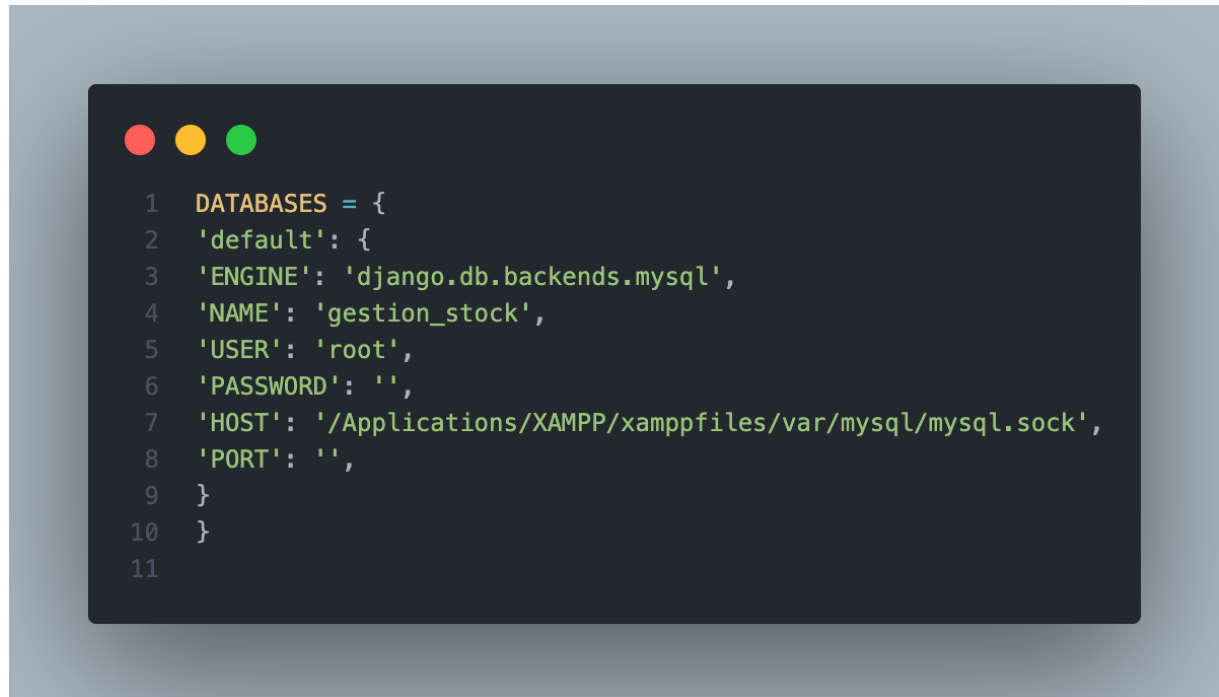


Figure 14 : Architecture de la base de données

Dans notre projet, nous avons utilisé le système de gestion de base de données MySQL grâce à ses performances, sa fiabilité et sa facilité d'intégration avec le framework Django.

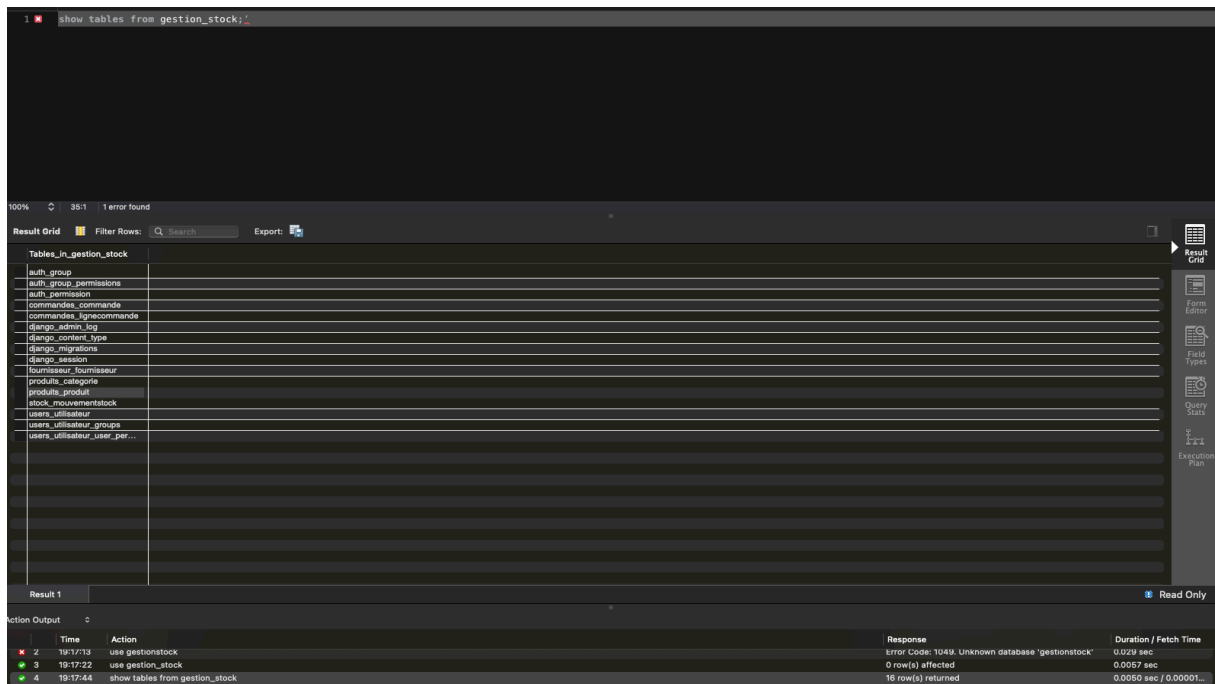


Figure 15 : Base de données MySQL

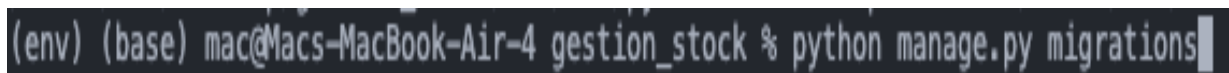
La base de données contient plusieurs tables importantes permettant de gérer efficacement les informations du système :

- table des produits,
- table des catégories,
- table des fournisseurs,
- table des utilisateurs,
- table des mouvements de stock.

Les relations entre les différentes tables assurent une meilleure organisation des données et permettent d'éviter la duplication des informations. Elles facilitent également les opérations de recherche, de modification et de consultation des données.

Le système ORM de Django a été utilisé pour communiquer avec la base de données sans écrire directement les requêtes SQL. Cette approche simplifie le développement et améliore la sécurité de l'application.

Les migrations Django ont également été utilisées afin de créer et modifier automatiquement les tables de la base de données durant le développement du projet.

A terminal window screenshot showing the command to run Django migrations. The prompt is '(env) (base) mac@Macs-MacBook-Air-4' and the command is 'gestion_stock % python manage.py migrations'. The cursor is at the end of the command.

```
(env) (base) mac@Macs-MacBook-Air-4 gestion_stock % python manage.py migrations
```

Figure 16: Exécution des migrations Django

Interfaces Graphiques

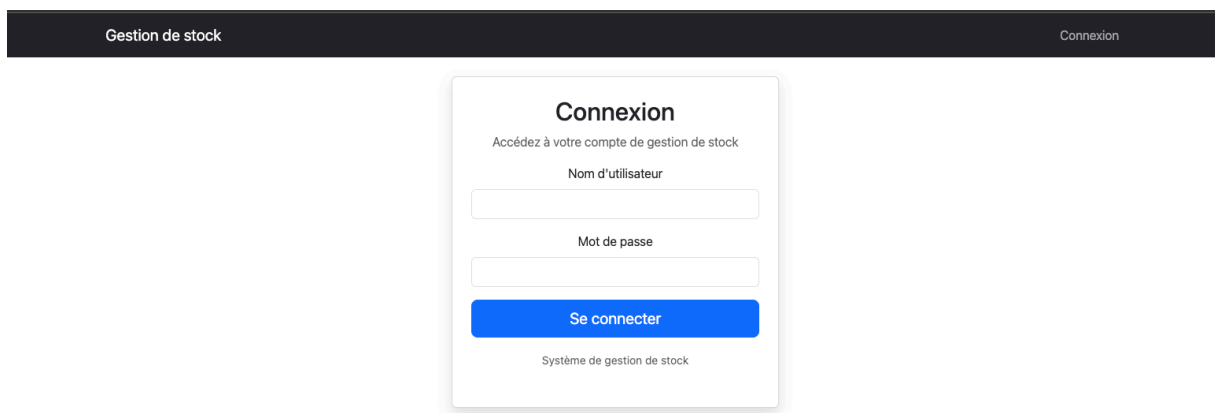
Dans cette partie, nous présentons les différentes interfaces développées dans l'application de gestion de stock. Ces interfaces permettent aux utilisateurs d'interagir facilement avec le système à travers des pages simples, modernes et ergonomiques.

Chaque interface a été conçue afin d'améliorer l'expérience utilisateur et faciliter l'exécution des différentes opérations de gestion du stock.

Interface de connexion

Cette interface permet aux utilisateurs de se connecter au système en utilisant leur nom d'utilisateur et leur mot de passe. Le système vérifie les informations saisies avant d'autoriser l'accès aux différentes fonctionnalités de l'application.

Cette étape garantit la sécurité des données et limite l'accès uniquement aux utilisateurs autorisés.



The screenshot displays a web application interface for a stock management system. At the top, a dark horizontal bar contains the text "Gestion de stock" on the left and "Connexion" on the right. Centered below this bar is a white login form with a subtle drop shadow. The form is titled "Connexion" in bold. Below the title is the instruction "Accédez à votre compte de gestion de stock". The form contains two input fields: "Nom d'utilisateur" and "Mot de passe". Below these fields is a prominent blue button labeled "Se connecter". At the bottom of the form, the text "Système de gestion de stock" is displayed.

Figure 17: Interface de connexion

Tableau de bord

Le tableau de bord représente l'interface principale du système. Il affiche une vue générale des informations importantes liées au stock.

Cette interface permet notamment d'afficher :

- le nombre total des produits,
- les mouvements de stock récents,
- les alertes de stock faible,
- les statistiques principales.

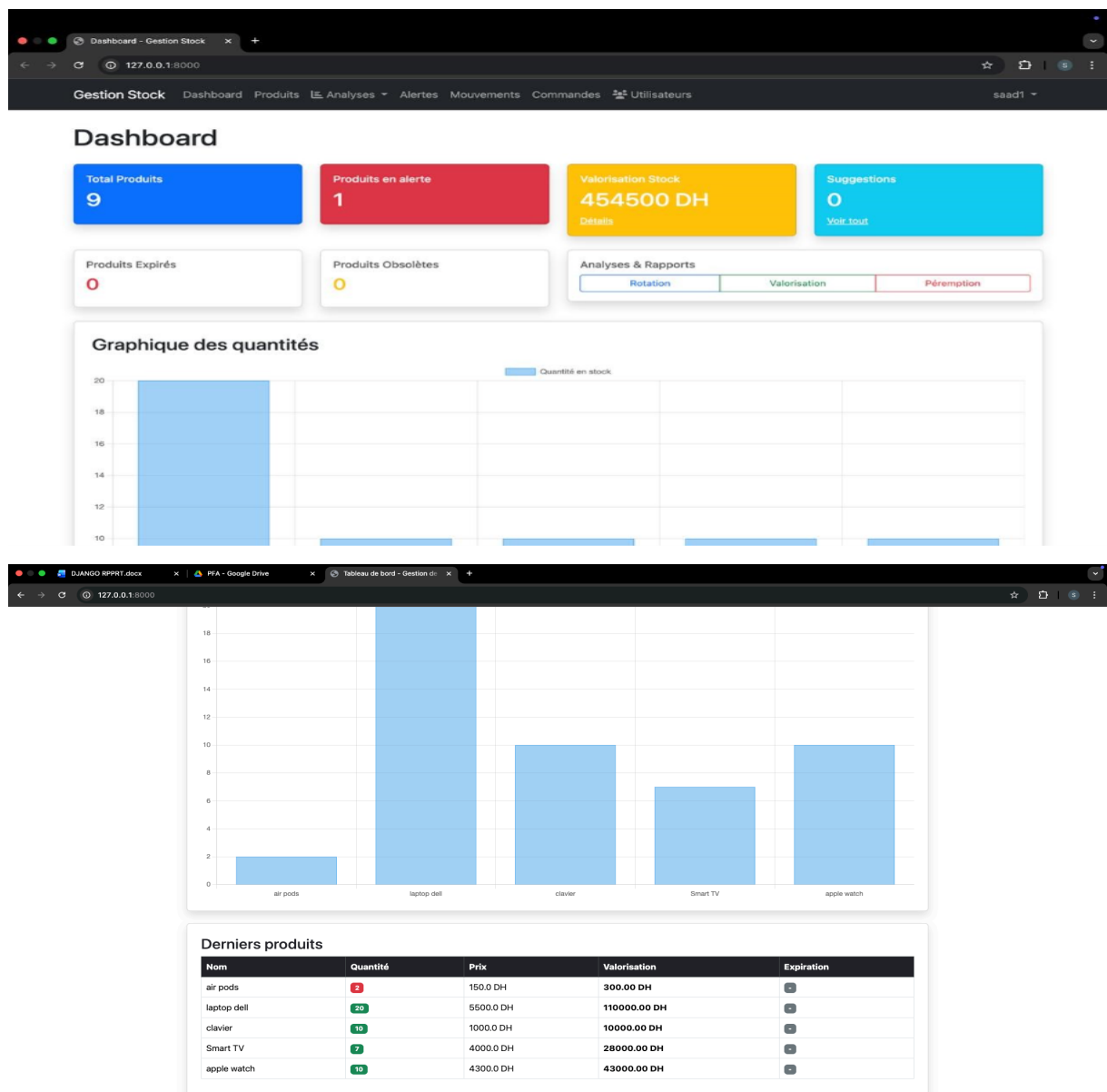


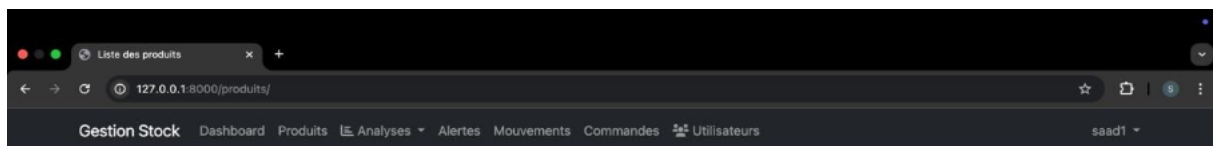
Figure 18 : Tableau de bord du système

Gestion des produits

Cette interface permet de gérer l'ensemble des produits disponibles dans le système. Les utilisateurs peuvent ajouter, modifier, supprimer ou consulter les produits enregistrés.

Chaque produit contient plusieurs informations importantes :

- nom,
- catégorie,
- quantité,
- prix,
- fournisseur.



Liste des produits

Ajouter un produit

Nom	Quantité	Prix	Catégorie	Fournisseur	Actions
laptop hp	20	4000.0 DH	informatique	Hp	Modifier Supprimer
Casque	25	300.0 DH	informatique	razer	Modifier Supprimer
iphone 15	15	7500.0 DH	informatique	apple	Modifier Supprimer
souris	1	1500.0 DH	informatique	logitech	Modifier Supprimer
Galaxy S25	10	5000.0 DH	informatique	samsung	Modifier Supprimer
apple watch	10	4300.0 DH	informatique	apple	Modifier Supprimer
Smart TV	10	4000.0 DH	électroménager	samsung	Modifier Supprimer
clavier	10	1000.0 DH	informatique	razer	Modifier Supprimer
laptop dell	20	5500.0 DH	informatique	Dell	Modifier Supprimer

Gestion de stock

Tableau de bord

Produits

Analyses

Alertes

Mouvements

Commandes

Utilisateurs

saad

Formulaire Produit

Nom du produit:

Description:

Quantité en stock:

Seuil d'alerte:

Code-barres:

Prix unitaire (DH):

Date de péremption:

jj/mm/aaaa

Date de péremption du produit

Statut d'obsolescence:

Actif

Date d'obsolescence:

jj/mm/aaaa

Date marquant l'obsolescence

Catégorie:

Fournisseur:

Enregistrer

Retour

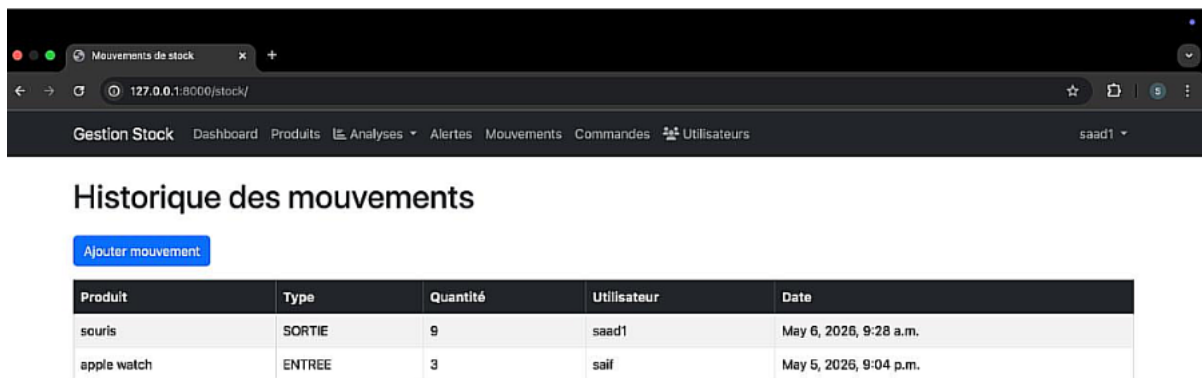
Figure 19: Gestion des produits

Gestion des mouvements de stock

Cette interface permet d'assurer le suivi des entrées et sorties de stock. Chaque mouvement enregistré contient des informations détaillées permettant une meilleure traçabilité des opérations réalisées.

Les informations affichées sont :

- type de mouvement,
- quantité,
- date,
- produit concerné.



Produit	Type	Quantité	Utilisateur	Date
souris	SORTIE	9	saad1	May 6, 2026, 9:28 a.m.
apple watch	ENTREE	3	saif	May 5, 2026, 9:04 p.m.

Figure 20 : Gestion des mouvements de stock

Tests

Tests fonctionnels

Les tests fonctionnels ont été réalisés afin de vérifier le bon fonctionnement de toutes les fonctionnalités développées dans l'application.

Les fonctionnalités testées sont :

- connexion utilisateur,
- ajout de produit,
- modification des produits,
- suppression des produits,
- gestion des mouvements,

Les résultats obtenus montrent que le système répond correctement aux besoins définis dans le cahier des charges.

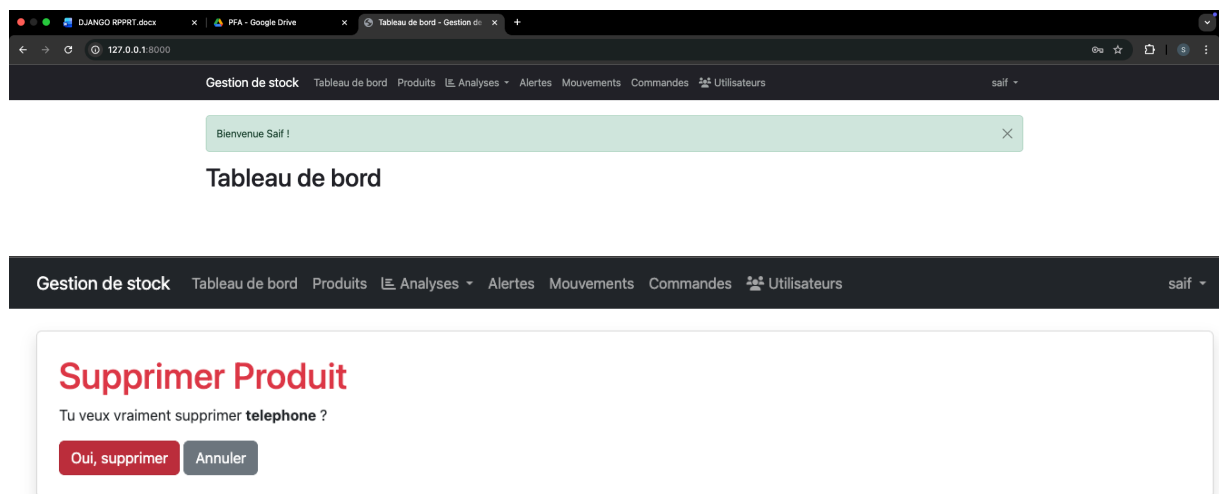


Figure 21 : Test fonctionnel

Tests de sécurité

Les tests de sécurité permettent de protéger les données et empêcher les accès non autorisés au système.

Plusieurs mécanismes de sécurité ont été utilisés :

- authentification Django,
- protection CSRF,
- validation des formulaires,
- contrôle d'accès aux pages sécurisées.

Ces mécanismes améliorent considérablement la sécurité de l'application et protègent les informations des utilisateurs.

A screenshot of a code editor with a dark background and light-colored text. The code is written in Python and represents a Django login view. It includes imports for Django shortcuts, authentication, messages, models, forms, and decorators. The `login_view` function checks if the request method is POST, retrieves the username and password, attempts authentication, and then either logs the user in and redirects based on their role or displays an error message.

```
1 from django.shortcuts import render, redirect, get_object_or_404
2 from django.contrib.auth import authenticate, login, logout
3 from django.contrib import messages
4 from .models import Utilisateur
5 from .forms import UtilisateurCreationForm, UtilisateurUpdateForm
6 from .decorators import admin_or_gestionnaire_required
7
8
9 def login_view(request):
10     if request.method == 'POST':
11         username = request.POST.get('username')
12         password = request.POST.get('password')
13
14         user = authenticate(request, username=username, password=password)
15
16         if user is not None:
17             login(request, user)
18             messages.success(request, f'Bienvenue {user.first_name or user.username} !')
19             if user.role == 'EMPLOYE':
20                 return redirect('liste_produits')
21             return redirect('home')
22         else:
23             messages.error(request, 'Nom d\'utilisateur ou mot de passe incorrect.')
24
25     return render(request, 'users/login.html')
```

Figure 22: Sécurisation des accès utilisateurs

Tests de performance

Les tests de performance ont été réalisés afin d'évaluer la rapidité et la stabilité du système pendant son utilisation.

Les résultats montrent :

- un chargement rapide des pages,
- une bonne fluidité de navigation,
- une gestion efficace des données,
- une stabilité correcte du serveur.

Le framework Django offre de bonnes performances pour les applications de gestion grâce à son architecture optimisée.

Chapitre 5 : Conclusion

Conclusion générale

Au terme de ce projet, nous avons réussi à concevoir et développer une application web de gestion de stock en utilisant le framework Django et le système de gestion de base de données MySQL.

L'objectif principal de ce système était de faciliter la gestion des produits, des fournisseurs, des utilisateurs ainsi que des mouvements de stock à travers une interface simple et moderne.

Durant la réalisation du projet, plusieurs fonctionnalités ont été développées avec succès, notamment :

- la gestion des utilisateurs,
- la gestion des produits,
- la gestion des catégories,
- la gestion des fournisseurs,
- la gestion des mouvements de stock,
- l'authentification et la sécurité des accès.

Le système développé permet d'améliorer l'organisation des données et de simplifier les opérations de gestion du stock. Grâce à l'utilisation du framework Django, le développement de l'application a été plus structuré, sécurisé et maintenable.

Ce projet nous a également permis d'acquérir plusieurs compétences techniques importantes dans le domaine du développement web, telles que :

- le développement backend avec Python et Django,
- la gestion des bases de données MySQL,
- la création d'interfaces web,
- l'utilisation des templates Django,
- la manipulation des formulaires et des opérations CRUD.

Malgré les résultats obtenus, certaines difficultés ont été rencontrées durant le développement du projet, notamment :

- la configuration de la base de données MySQL,
- la gestion des migrations Django,
- la résolution des erreurs de routage et de templates,
- l'organisation des différentes applications Django.

Cependant, ces difficultés ont permis de renforcer nos connaissances techniques et d'améliorer notre capacité à résoudre les problèmes rencontrés durant le développement des applications web.

Finalement, ce projet représente une expérience enrichissante aussi bien sur le plan technique que pratique, et constitue une étape importante dans notre formation en génie informatique.

Perspectives

Dans le futur, plusieurs améliorations peuvent être apportées à cette application afin d'ajouter de nouvelles fonctionnalités et améliorer davantage les performances du système.

Parmi les améliorations envisagées :

- développement d'une application mobile permettant la gestion du stock depuis un téléphone,
- intégration d'un système de notifications pour les alertes de stock faible,
- ajout d'un tableau de bord plus avancé avec statistiques et graphiques,
- intégration d'un système d'intelligence artificielle pour prévoir les besoins en stock,
- ajout d'un chatbot intelligent pour assister les utilisateurs,
- génération automatique des rapports PDF et Excel,
- amélioration de la sécurité et des permissions des utilisateurs,
- ajout d'un système de sauvegarde automatique des données,
- utilisation d'API modernes pour améliorer la communication entre les services.

Ces améliorations permettront de rendre le système plus intelligent, plus rapide et plus adapté aux besoins des entreprises modernes.

Bibliographie

Les ressources suivantes ont été utilisées durant la réalisation de ce projet :

Sites web

- <https://www.django.com>
- <https://www.Mysql.com>
- <https://www.bootstrap.com>
- <https://www.w3schools.com>